MiniClaw: An Always-on Personal Assistant

Background

A local assistant should remain useful after the user stops typing. It should remember users and conversations, run planned work at the right time, check for new items when needed, and keep the user in control of actions that affect the user or another system.

MiniClaw centers on a long-running gateway. A CLI sends messages and commands to it. The gateway stores users, sessions, recurring schedules, jobs, heartbeat settings, and delivery records; calls a real Agent executor through an adapter; and sends approved results through a local outbox. After a clean restart, the same work and status must still be available.

Long-running assistants also need a clear boundary between internal state and actions visible outside the system. Saving a record changes only local state, while sending a message, writing to a target, or triggering another visible action can affect the user or another system.

Basic requirements - 70 points

Basic is required and covers the complete workflow.

Gateway and CLI channel - 15 points

Run a persistent gateway with `run` and provide one adapter-backed CLI for messages, schedules, status queries, and approvals.

Identity and state - 10 points

Keep users, sessions, recurring schedules, jobs, and delivery state across a clean restart.

Recurring schedules and jobs - 10 points

Save a simple recurring schedule, create a job at each due occurrence without a new message, and record its status and result.

Heartbeat monitor - 10 points

Let the user enable or disable a fixed-interval check and set one check task. Wake the Agent at each interval, stay quiet when there is nothing to handle, and require approval before sending a notification.

Approval, outbox, and delivery - 10 points

Save internal state directly, require approval for visible actions, place an approved result in the local outbox before delivery, and never deliver a denied action.

Agent executor and one extension - 10 points

Call a real Agent executor through an adapter and use one Skill or MCP tool whose output changes the result. An API failure produces a failed job, not a fabricated result.

Tests, evidence, and documentation - 5 points

Provide `test`, fixed local data, a structured run record, and short instructions for running and checking the system.

The live run uses `run` to show a schedule, a due job, the Agent result, extension output, a Heartbeat check, approval, delivery, and state after restart. Set time and cost limits for model/API calls.

Advanced requirements - 30 points

Advanced work is optional and reuses the same gateway and task model.

Second channel: GUI, TUI, or Web - 10 points

Add one second channel in a GUI, TUI, or Web interface. It must use its own adapter and reach the same users, sessions, schedules, and jobs.

Reliable background execution - 4 points

Add visible cancellation and timeout states, persisted retry or recovery handling, and protection against duplicate job execution.

Hot-swappable workflow extension - 4 points

Replace the Basic Skill or MCP tool through configuration without changing gateway routing, and show that the replacement changes the Agent result.

Long-session context compaction - 4 points

Compact a long conversation while preserving its goal and pending work, then resume the same session correctly.

Schedule and task lifecycle management - 4 points

Let users list, pause, resume, edit, and cancel recurring schedules and Heartbeat settings. Changes must survive a clean restart and affect future runs without corrupting existing jobs.

Runtime monitoring and audit trail - 4 points

Provide a simple way to inspect recent triggers, jobs, deliveries, and failures, with timestamps and IDs that connect each record to the same run.

Submission and implementation requirements

Submit one runnable repository. It must include:

- Clear setup instructions and working `run` and `test` entry points.
- The configuration and sample data needed to reproduce the main workflow.
- A short architecture note and structured evidence from one complete run.
- A list of the Advanced items implemented, with a brief explanation of how each can be checked.

All claimed features must work in the submitted system and use the same gateway and persistent state.